



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Departamento de Matemática

Manual cluster_tools v1.1
Proyecto modelación minera
Semestre 2025-1

Contents

1	Plotting	2
1.1	Plotting 3D	2
1.1.1	plot_mina_3D	2
1.1.2	plot_precedencias_verticales_3D	2
1.2	Plotting 2D	3
1.2.1	plot_fase_banco	3
1.2.2	animate_fase_banco	4
2	Clusterización	5
2.1	Funciones previas	5
2.1.1	Hadamard_Product_Sparse	5
2.1.2	Calculate_Adjency_Matrix	5
2.1.3	Precedencia_Fase_Banco	6
2.1.4	Calculate_Vertical_Adjency_Matrix	6
2.1.5	Calculate_Similarity_Matrix	7
2.1.6	Find_Most_Similar_Adjacent_Clusters	8
2.1.7	Centros_Clusters	8
2.1.8	Clusters_Vecinos	9
2.1.9	Precedencias_Clusters_Agend	9
2.1.10	Precedencias_Clusters_Angle	10
2.1.11	Z_min	10
2.1.12	Puntos_Iniciales	11
2.2	Clustering	11
2.2.1	Clustering_Tabesh	11
2.2.2	Clustering_mina	12
2.3	Shape Refinement	13
2.3.1	Find_Corner_Blocks_Tabesh	13
2.3.2	Shape_Refinement_Tabesh	13
2.3.3	Shape_Refinement_Mod	14
3	Métricas	14
3.1	Métricas consideradas por Tabesh	14
3.1.1	Rock_Unity	14
3.1.2	Destination_Dilution_Factor	15
3.1.3	Coefficient_Variation	15
4	Paralelización	15
4.1	Funciones de paralelización del proceso de clusterización	15
4.1.1	clustering_wrapper	15
4.1.2	Clustering_parallel	15

1 Plotting

1.1 Plotting 3D

1.1.1 plot_mina_3D

```
cluster_tools.plot_mina_3D(mina, column_hue='tipomineral', params=dict())
```

Realiza un gráfico de la mina en 3D.

Parámetros:

mina : pandas.DataFrame

Dataframe de la mina.

column_hue : string

Columna del dataframe respecto al cual se colorean los bloques de la mina.

params : dict

Diccionario con los parámetros opcionales de entrada del gráfico.

Opción	Descripción	Valor predeterminado	Tipo de dato
width	Ancho del gráfico	900	int
height	Altura del gráfico	800	int
opacity_blocks	Opacidad de los bloques graficados. Número real entre 0 y 1	1	float
puntos	Puntos extra graficados. Input de la forma $[(x_i, y_i, z_i)]_{i=1}^n$	<code>list()</code>	list
curvas	Curvas. Input de la forma $[(x_i)_{i=1}^n, (y_i)_{i=1}^n, (z_i)_{i=1}^n]$	<code>list()</code>	list
elipses	Elipses. Input de la forma $[(a_i, b_i, \alpha_i, x_i, y_i, z_i)]_{i=1}^n$	<code>list()</code>	list
conos	Conos. Input de la forma $[(a_i, b_i, h_i, \alpha_i, x_i, y_i, z_i)]_{i=1}^n$	<code>list()</code>	list
id_bloques	ID de bloques destacados. Input de la forma $set\{id_i\}_{i=1}^n$	<code>set()</code>	set
mina_rellena	Mina rellena, en caso de especificar IDs fuera de la mina.	<code>pandas.DataFrame()</code>	pandas.DataFrame
z_ratio	Factor de escala del eje z	1	float

Table 1: Opciones admitidas por `params`.

1.1.2 plot_precedencias_verticales_3D

```
cluster_tools.plot_precedencias_verticales_3D(  
    fases_bancos, column_hue='cluster',  
    precedence_option=True, arcs={},  
    width=1000, height=800, z_ratio=5)
```

Gráfico 3D de las precedencias verticales entre dos fases-bancos.

Parámetros:

fases_bancos : list

Lista de dos fases-bancos que tienen precedencias verticales. Formato:

fases_bancos = [*fase_banco_lower*, *fase_banco_upper*]

column_hue : string

Columna del dataframe respecto al cual se colorean los bloques de la mina.

precedence_option : bool

Habilita el uso de arcos para graficar las precedencias. Si es False, simplemente se grafican las dos fases-bancos.

arcs : dict()

Diccionario con las precedencias verticales. Estructura creada en *aux_tools*.

width : int

Ancho del gráfico.

height : int

Altura del gráfico.

z_ratio : float

Factor de escala del eje *z*.

1.2 Plotting 2D

1.2.1 *plot_fase_banco*

```
cluster_tools.plot_fase_banco(FaseBanco, column_hue='cut', text_hue=None, params=dict()):
```

Función para graficar una fase-banco en 2D.

Parámetros:

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada *z*).

column_hue : string

Columna del dataframe respecto al cual se colorean los bloques de la fase-banco.

text_hue : string

Columna del dataframe respecto al cual se muestra el texto de los bloques de la fase-banco. Opcional en caso de querer usar un texto distinto al de *column_hue*.

params : dict()

Diccionario con las opciones adicionales de plotting.

Opción	Descripción	Valor predeterminado	Tipo de dato
<code>BlockWidthX</code>	Ancho de bloque (eje x)	10	int
<code>BlockWidthY</code>	Altura de bloque (eje y)	10	int
<code>x_size</code>	Ancho del gráfico.	10	int
<code>y_size</code>	Altura del gráfico	10	int
<code>cmap</code>	Mapa de colores. Vea los colores disponibles por matplotlib.	'plasma'	string
<code>show_block_label</code>	Indica si se muestra texto encima de los bloques.	True	bool
<code>show_grid</code>	Indica si se muestra una grilla en el gráfico	True	bool
<code>show_legend</code>	Indica si se muestra la leyenda del gráfico.	True	bool
<code>dpi</code>	Resolución del gráfico.	100	int
<code>destacar_bloques</code>	ID de bloques destacados. Input de la forma $set\{id_i\}_{i=1}^n$	set()	set
<code>puntos</code>	Puntos graficados. Input de la forma $[(x_i, y_i)]_{i=1}^n$	list()	list
<code>flechas</code>	Flechas graficados. Input de la forma $[((x_i^1, y_i^1), (x_i^2, y_i^2))]_{i=1}^n$	list()	list
<code>sectores</code>	Sectores. Input de la forma $[(x_1^i, x_2^i, x_3^i, x_4^i)]_{i=1}^n$	list()	list
<code>precedencias</code>	Diccionario de precedencias horizontales. Estructura entregada por 'Precedencias_Clusters_Agend'.	dict()	dict
<code>centros</code>	Centros (de los clusters). Estructura entregada por 'Centros_Clusters'.	dict()	dict
<code>elipse</code>	Parametros de elipse. Input de la forma $[(a^i, b^i, x^i, y^i, \alpha^i)]_{i=1}^n$	list()	list
<code>guardar_imagen</code>	Indica si se quiere guardar la imagen en un directorio entregado por 'path'.	False	bool
<code>path</code>	Directorio donde se guarda la imagen.	"	string

Table 2: Opciones admitidas por `params`.

1.2.2 `animate_fase_banco`

`cluster_tools.animate_fase_banco(history, column_hue='cluster', params=dict(), save_path=None, fps=)`

Función para graficar una fase-banco en 2D.

Parámetros:

history : list de pandas.DataFrame

Lista de DataFrames con la evolución del clusterizado en cada paso.

column_hue : string

Columna del dataframe respecto al cual se colorean los bloques de la fase-banco.

params : dict()

Diccionario con las opciones adicionales de plotting. Es el mismo diccionario entregado a la función `plot_fase_banco` (vea tabla 2), al que se le agregan las opciones 'xsize', 'ysize' y 'dpi' para configurar el tamaño y resolución del gráfico.

2 Clusterización

2.1 Funciones previas

2.1.1 Hadamard_Product_Sparse

`cluster_tools.Hadamard_Product_Sparse(A,B)`

Función auxiliar para calcular el producto de Hadamard (componente a componente) entre una matriz sparse (A) y una matriz densa (B), las cuales tienen las mismas dimensiones. Devuelve $A : B$.

Parámetros:

A : sparse.csr_matrix
Matriz input.

B : array_like
Matriz input.

Retorno:

resultado_sparse : sparse.csr_matrix
Matriz output, igual a $A : B$.

2.1.2 Calculate_Adgency_Matrix

`cluster_tools.Calculate_Adgency_Matrix(FaseBanco, BlockWidthX, BlockWidthY, Sectores=[])`

Calcula la matriz de adyacencia de los bloques de una fase-banco dada. La coordenada (i, j) es 1 si el bloque i (de posición i en el dataframe) es adyacente al bloque j , y 0 si no. Devuelve una matriz sparse csr.

Parámetros:

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

BlockWidthX : int

Entero positivo que indica el ancho de los bloques (en eje x).

BlockWidthY : int

Entero positivo que indica la altura de los bloques (en eje y).

Sectors : list

Es una lista de 4-tuplas que delimita sectores de la fase-banco. Es de la forma

$$Sectors = [(x_1^1, x_2^1, x_3^1, x_4^1), (x_1^2, x_2^2, x_3^2, x_4^2), \dots, (x_1^n, x_2^n, x_3^n, x_4^n)]$$

Cada sector es un cuadrilátero cuyos vértices son $(x_1^i, x_2^i, x_3^i, x_4^i) \in \mathbb{R}^2 \times \mathbb{R}^2 \times \mathbb{R}^2 \times \mathbb{R}^2$, donde

- (a) x_1^i es el vértice inferior izquierdo,
- (b) x_2^i es el vértice inferior derecho,
- (c) x_3^i es el vértice superior derecho,
- (d) x_4^i es el vértice superior izquierdo.

Retorno:

adjency_matrix : sparse.csr_matrix
Matriz de adyacencia de la fase-banco.

2.1.3 Precedencia_Fase_Banco

```
cluster_tools.Precedencia_Fase_Banco(df)
```

Calcula las relaciones de precedencia vertical en una mina.

Parámetros:

df : pandas.DataFrame

DataFrame de una mina con las columnas 'fase', 'banco', 'z', 'x' y 'y'.

Retorno:

resultados : list de tuples

Lista de tuplas ((fase_superior, banco_superior), (fase_inferior, banco_inferior)) que cumplen la relación de precedencia vertical.

2.1.4 Calculate_Vertical_Adjency_Matrix

```
cluster_tools.Calculate_Vertical_Adjency_Matrix(df_sup, df_inf, BlockWidthX, BlockWidthY,  
cluster_col='cluster')
```

Calcula la matriz de adyacencia vertical entre clusters de dos fases-banco consecutivas de la mina.

Se asume que 'df_sup' corresponde a una fase superior (más alta en Z) y 'df_inf' a la fase inmediatamente inferior.

La función identifica bloques adyacentes verticalmente según su distancia euclidiana en el plano (x, y), normalizada por el tamaño del bloque, y luego construye una matriz de precedencia entre clusters de ambas fases.

Parámetros:

df_sup : pandas.DataFrame

DataFrame que representa la fase superior. Debe contener las columnas 'x', 'y' y 'cluster_col'.

df_inf : pandas.DataFrame

DataFrame que representa la fase inferior. Debe contener las columnas 'x', 'y' y 'cluster_col'.

BlockWidthX : float

Ancho de los bloques en la dirección X.

BlockWidthY : float

Ancho de los bloques en la dirección Y.

cluster_col : str

Nombre de la columna que identifica el cluster al que pertenece cada bloque. Por defecto es 'cluster'.

Retorno:

A_clusters : numpy.ndarray tipo int

Matriz binaria de tamaño (n_sup, n_inf), donde 'n_sup' es el número de clusters en la fase superior y 'n_inf' es el número de clusters en la fase inferior. La entrada (i, j) es 1 si hay al menos un bloque del cluster 'i' en la fase superior que es adyacente a un bloque del cluster 'j' en la fase inferior.

2.1.5 Calculate_Similarity_Matrix

`cluster_tools.Calculate_Similarity_Matrix(FaseBanco, params = dict())`

Calcula la similaridad entre los bloques de la fase-banco, de acuerdo a los criterios de distancia, ley, destino, tipo de roca y/o dirección de minería. Devuelve una matriz densa.

Se calcula de la siguiente manera:

$$S_{ij} = \begin{cases} \frac{R_{ij} \cdot T_{ij}}{D_{ij}^{W_D} \cdot G_{ij}^{W_G} \cdot M_{ij}^{W_M}} & \text{si } i \neq j \\ 0 & \text{si } i = j \end{cases} \quad (1)$$

donde

- R_{ij} es una medida de similitud que compara el tipo de roca de los bloques i y j . Se define como

$$R_{ij} = \begin{cases} 1 & \text{si los bloques } i \text{ y } j \text{ son del mismo tipo de roca} \\ r & \text{en caso contrario} \end{cases} \quad (2)$$

con $r \in [0, 1]$.

- T_{ij} es una medida de similitud que compara el destino de los clusters i y j . Se define como

$$T_{ij} = \begin{cases} 1 & \text{si los bloques } i \text{ y } j \text{ comparten destino} \\ t & \text{en caso contrario} \end{cases} \quad (3)$$

con $t \in [0, 1]$

- D_{ij} es una medida de disimilitud, el cual es la distancia normalizada entre los bloques i y j . Se define como

$$D_{ij} = \frac{\|(x_i, y_i) - (x_j, y_j)\|_2}{D_{max}} \quad (4)$$

donde (x_i, y_i) son las coordenadas del cluster i y D_{max} es la mayor distancia entre todos los bloques de la fase-banco. Además, $W_D \geq 0$ es el peso asociado a esta medida.

- G_{ij} es una medida de disimilitud, el cual compara las leyes de los bloques. Se suele considerar un parámetro $\varepsilon > 0$ pequeño que acota la medida inferiormente para evitar medidas nulas. Se define como

$$G_{ij} = \begin{cases} |G_i - G_j| & \text{si } |G_i - G_j| \geq \varepsilon_G \\ \varepsilon_G & \text{si } |G_i - G_j| < \varepsilon_G \end{cases} \quad (5)$$

donde G_i es la ley del bloque i . Además, $W_G \geq 0$ es el peso asociado a esta medida.

- M_{ij} es una medida de disimilitud para favorecer que la clusterización tenga formas coherentes con una dirección de minería en particular. Previo al cálculo del índice de similitud, se deben entregar un par de puntos $P1, P2 \in \mathbb{R}^2$ que definen una dirección de minería, y el índice se calcula de la siguiente forma:

$$\begin{aligned} M_i^1 &= \|(x_i, y_i) - P1\|^2, & M_i^2 &= \|(x_i, y_i) - P2\|^2 \\ M_i &= \text{sign}((M_i^1)^2 - (M_i^2)^2) \sqrt{|(M_i^1)^2 - (M_i^2)^2|} \\ M_{ij} &= \begin{cases} |M_i - M_j| & \text{si } |M_i - M_j| \geq \varepsilon_M \\ \varepsilon_M & \text{si } |M_i - M_j| < \varepsilon_M \end{cases} \end{aligned} \quad (6)$$

Además, $W_M \geq 0$ es el peso asociado a esta medida.

Parámetros:

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

Opción	Descripción	Valor predeterminado
peso_distancia	W_D	2
peso_ley	W_G	0
tolerancia_ley	ε_G	0.001
peso_direccion_mineria	W_M	0.25
tolerancia_direccion_mineria	ε_M	0.001
vector_mineria	$(P1, P2)$	None
penalizacion_destino	$1 - t$	0.9
penalizacion_roca	$1 - r$	0.9

Table 3: Opciones admitidas por *params*.

params : dict

Diccionario con los parámetros de entrada de la matriz de similaridad. Utiliza los criterios de la tabla 3.

Si se requiere anular un criterio, basta con asignar el peso o penalización correspondiente al valor 0.

Retorno:

similarity_matrix : array_like

Matriz de similaridad de los bloques de la fase-banco.

2.1.6 Find_Most_Similar_Adjacent_Clusters

`cluster_tools.Find_Most_Similar_Adjacent_Clusters(AdjacencyMatrix, SimilarityMatrix)`

Busca los clusters adyacentes más similares entre sí. Devuelve una lista con los índices de los clusters más similares y, si no encuentra pares similares, devuelve None.

Parámetros:

AdjacencyMatrix : sparse.csr_matrix

Matriz sparse entregada por `cluster_tools.Adjacency_Matrix`.

SimilarityMatrix : array_like

Matriz sparse entregada por `cluster_tools.Similarity_Matrix`.

Retorno:

$[i,j]$: list

Índice de los clusters adyacentes más similares (de acuerdo a los índices de la matriz de adyacencia/similaridad).

2.1.7 Centros_Clusters

`cluster_tools.Centros_Clusters(FaseBanco)`

Calcula los centros geométricos de los clusters. Retorna un diccionario cuyas llaves son el ID del cluster y su valor son las coordenadas (x,y) de su centro.

Parámetros:

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

Retorno:

Centers : dict

Centros geométricos de los clusters de FaseBanco.

2.1.8 Clusters_Vecinos

`cluster_tools.Clusters_Vecinos(FaseBanco, Cluster, AdjacencyMatrix)`

Calcula los clusters vecinos de un clutstre dado en la fase banco, usando la matriz de adyacencia.

Parámetros:

FaseBanco : pandas.DataFrame

DataFrame de la fase banco. Debe contener la columna 'cluster'.

Cluster: int

ID del cluster para el cual se buscan los clusters vecinos.

AdjacencyMatrix : scipy.sparse matrix

Matriz de adyacencia.

Retorno:

clusters_vecinos list of int Lista (con posibles repeticiones) de los IDs de los clusters vecinos.

2.1.9 Precedencias_Clusters_Agend

`cluster_tools.Precedencias_Clusters_Agend(FaseBanco, vector_mineria, BlockWidthX=10, BlockWidthY=10, Distance_Option=True)`

Calcula las precedencias horizontales de los clusters de acuerdo a la dirección de minería dada. Devuelve un diccionario cuyas llaves son los índices de los clusters y sus valores son sus clusters inmediatamente precedentes.

Parámetros:

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

vector_mineria : tuple

2-Tupla (P_{inicio} , P_{final}) que indican el vector de dirección de minería.

BlockWidthX : int

Entero positivo que indica el ancho de los bloques (en eje x).

BlockWidthY : int

Entero positivo que indica la altura de los bloques (en eje y).

Distance_Option : bool

Opción de cálculo de distancia al punto de inicio. Si su valor es True, calcula la distancia al punto de inicio proyectando primero al vector de minería (Recomendado). Si su valor es False, calcula la distancia euclideana al punto de inicio.

Retorno:

precedencias_Clusters : dict

Diccionario cuyas llaves son los índices de los clusters y sus valores son sus clusters inmediatamente precedentes.

fase_banco : pandas.DataFrame

Dataframe de la fase-banco después de la clusterización. Agrega una nueva columna llamada 'cluster'.

New_centeres : dict

Diccionario con los centros de los clusters.

execution_time : float

Tiempo de ejecución.

2.1.10 Precedencias_Clusters_Angle

```
cluster_tools.Precedencias_Clusters_Angle(FaseBanco, vector_mineria,  
BlockWidthX=10, BlockWidthY=10, Distance_Option=True, Angle=0)
```

Calcula las precedencias horizontales de los clusters de acuerdo a la dirección de minería dada. Devuelve un diccionario cuyas llaves son los índices de los clusters y sus valores son sus clusters inmediatamente precedentes.

Es una modificación del algoritmo de precedencias previo, el cual calcula las precedencias restringiendo los clusters precedentes a un cono.

Parámetros:

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

vector_mineria : tuple

2-Tupla (P_{inicio} , P_{final}) que indican el vector de dirección de minería.

BlockWidthX : int

Entero positivo que indica el ancho de los bloques (en eje x).

BlockWidthY : int

Entero positivo que indica la altura de los bloques (en eje y).

Distance_Option : bool

Opción de cálculo de distancia al punto de inicio. Si su valor es True, calcula la distancia al punto de inicio proyectando primero al vector de minería (Recomendado). Si su valor es False, calcula la distancia euclídeana al punto de inicio.

Angle : float

Ángulo (en radianes) del cono de precedencias.

Retorno:

precedencias_Clusters : dict

Diccionario cuyas llaves son los índices de los clusters y sus valores son sus clusters inmediatamente precedentes.

fase_banco : pandas.DataFrame

Dataframe de la fase-banco después de la clusterización. Agrega una nueva columna llamada 'cluster'.

New_centeres : dict

Diccionario con los centros de los clusters.

execution_time : float

Tiempo de ejecución.

2.1.11 Z_min

```
cluster_tools.Z_min(mina, cono, min_area=None, debug=False)
```

Calcula la altura mínima desde el cual la base del cono supera un área mínima.

Parámetros:

mina : pandas.DataFrame

Dataframe de la mina.

cono : tupla

7-Tupla $(a, b, h, \alpha, x_c, y_c, z_c)$ que describen el cono.

- a, b son los semiejes de la elipse que forma la base del cono.
- h es la altura del cono.
- α es la orientación angular de la elipse.
- x_c, y_c es el centro de la base del cono.
- z_c es la altura a la cual está ubicada la base del cono.

min_area : float

Área mínima de la operación.

debug : bool

Si es True, se imprime información adicional.

Retorno:

z_min : float

Valor mínimo de z donde la sección horizontal del cono alcanza *min_area*.

2.1.12 Puntos Iniciales

```
cluster_tools.Puntos_Iniciales(mina, rampa, z_min=None, debug=False)
```

Obtiene los puntos iniciales de excavación a partir de una rampa dada, hasta un nivel mínimo.

Parámetros:

mina : pandas.DataFrame

Dataframe de la mina.

rampa : tupla de arrays

3-Tupla (X, Y, Z) con que describen la curva de la rampa.

z_min : float

Indica el nivel mínimo que alcanza la rampa.

debug : bool

Si es True, se imprime información adicional.

Retorno:

puntos_iniciales : list de tuplas

Lista de tuplas (x, y, z) con las coordenadas de los puntos iniciales de excavación.

2.2 Clustering

2.2.1 Clustering.Tabesh

```
cluster_tools.Clustering_Tabesh(FaseBanco, AdjacencyMatrix, SimilarityMatrix, params=dict(),  
animation_mode=False)
```

Realiza la rutina base de Clusterización (Tabesh 2013). Devuelve un dataframe de la fase-banco con la nueva columna 'cluster', que es el cluster al que pertenece cada bloque, el tiempo de ejecución y el número de clusters creados.

Parámetros:

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

AdjacencyMatrix : sparse.csr_matrix

Matriz sparse entregada por `cluster_tools.Adjacency_Matrix`.

SimilarityMatrix : array_like

Matriz sparse entregada por `cluster_tools.Similarity_Matrix`.

params : dict

Diccionario con los parámetros de entrada de la rutina de clusterización.

Opción	Descripción	Valor predeterminado
Average_Length_Cluster	Tamaño de cluster promedio deseado. Su valor debe ser menor a <code>Max_Length_Cluster</code>	30
Max_Length_Cluster	Tamaño máximo de cluster	35
Reset_Cluster_Index	Indica si se reinicia el índice de los clusters de 1 a $N_{clusters}$.	True
debug	Imprimir datos adicionales	False

Table 4: Opciones admitidas por *params*.

animation_mode : bool

Si es True, se guarda el historial completo del proceso de fusión de clusters, útil para animaciones o visualización.

Retorno:

fase_banco : pandas.DataFrame

Dataframe de la fase-banco después de la clusterización. Agrega una nueva columna llamada 'cluster'.

execution_time : float

Tiempo de ejecución.

N_clusters : int

Número de clusters creados.

history : list de pandas.DataFrame. (solo si *animation_mode* = True)

Historial del estado del DataFrame 'fase_banco' después de cada fusión, para reconstrucción del proceso paso a paso.

2.2.2 Clustering_mina

`Clustering_mina(mina, cm=2, cr=0.25, cp=10, P=4, R=0.85, alpha_ley_corte=0, options=dict())`

Realiza un proceso jerárquico de clusterización sobre un bloque de mina dividido por fases y bancos, agrupando los bloques en clusters con criterios geológicos, económicos y geométricos.

Este método clasifica bloques de una mina en clústeres operacionales, aplicando restricciones de ley de corte, precedencias operativas y formas geométricas. Se realiza por fases y bancos, permitiendo un refinamiento posterior de la forma de los clústeres y el control sobre el tamaño y distribución espacial de los mismos. Se generan métricas geológicas y relaciones de precedencia entre clusters para planificaciones futuras.

Parámetros:

mina : pandas.DataFrame

DataFrame que contiene los bloques mineros. Debe contener las pertenencias a fases y bancos en columnas llamadas 'fase' y 'banco'.

cm : float

Costo por tonelada de material extraído.

cr : float

Costo por tonelada de material refinado.

cp : float

Costo por tonelada de material enviado a planta.

P : float

Precio del cobre en dólares por libra.

R : float (entre 0 y 1)

Recuperación metalúrgica.

alpha_ley_corte : float (entre 0 y 1)

Parámetro de interpolación entre la ley de corte marginal y la ley de corte crítica para la elección de la ley de corte. Si es 0, se escoge la ley de corte marginal, y si es 1, se escoge la ley de corte crítica.

options : dict

Diccionario con los parámetros de clusterización. Vea tabla 5.

2.3 Shape Refinement

2.3.1 Find_Corner_Blocks_Tabesh

`cluster_tools.Find_Corner_Blocks_Tabesh(FaseBanco, AdjencyMatrix)`

Busca los bloques esquina de FaseBanco de acuerdo al criterio Tabesh 2013. Devuelve un diccionario cuyas llaves son el 'id' de los bloques esquina, y cuyos valores son los clusters con los que es vecino.

Parámetros:

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

AdjencyMatrix : sparse.csr_matrix

Matriz sparse entregada por `cluster_tools.Adjency_Matrix`.

2.3.2 Shape Refinement_Tabesh

`cluster_tools.Find_Corner_Blocks_Tabesh(FaseBanco, AdjencyMatrix,
Min_Cluster_Length=10, Iterations_PostProcessing=5, Reset_Clusters_Index=False)`

Refina la forma de los clusters de FaseBanco utilizando el criterio de Tabesh 2013. Devuelve el dataframe de la fase-banco, el tiempo de ejecución y el número de clusters finales.

Parámetros

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

AdjencyMatrix : sparse.csr_matrix

Matriz sparse entregada por `cluster_tools.Adjency_Matrix`.

Min_Cluster_Lenght : int

Tamaño mínimo deseado de los clusters.

*Iterations_
Post_Processing* : int

Número de iteraciones del algoritmo de refinamiento.

*Reset_
Clusters_Index* : bool

Indica si se reinicia el índice de los clusters (False recomendado).

2.3.3 Shape_Refinement_Mod

```
cluster_tools.Find_Corner_Blocks_Tabesh(FaseBanco, AdjencyMatrix,  
Min_Cluster_Length=10, Iterations_PostProcessing=5, Reset_Clusters_Index=False)
```

Refina la forma de los clusters de FaseBanco. Es una modificación del algoritmo de refinamiento de Tabesh (Shape_Refinement_Tabesh) el cual considera más casos de bloques esquina a costa de ser un algoritmo secuencial (depende del ordenamiento de los clusters). Devuelve el dataframe de la fase-banco, el tiempo de ejecución y el número de clusters finales.

Parámetros

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

AdjencyMatrix : sparse.csr_matrix

Matriz sparse entregada por `cluster_tools.Adjency_Matrix`.

Min_Cluster_Lenght : int

Tamaño mínimo deseado de los clusters.

*Iterations_
Post_Processing* : int

Número de iteraciones del algoritmo de refinamiento.

*Reset_
Clusters_Index* : bool

Indica si se reinicia el índice de los clusters (False recomendado).

3 Métricas

3.1 Métricas consideradas por Tabesh

3.1.1 Rock_Unity

```
cluster_tools.Rock_Unity(FaseBanco)
```

Calcula la homogeneidad en el tipo de roca de los clusters de la fase-banco (Rock Unity). Devuelve el promedio de Rock Unity de los clusters y una lista con los valores individuales de Rock Unity por cada cluster.

Parámetros

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

3.1.2 Destination_Dilution_Factor

```
cluster_tools.Destination_Dilution_Factor(FaseBanco)
```

Calcula la homogeneidad del destino de los clusters de la fase-banco, pesado por el tonelaje de cada bloque (DDF). Devuelve el promedio del DDF de la fase-banco y una lista con los valores individuales de DDF de cada cluster.

Parámetros

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

3.1.3 Coefficient_Variation

```
cluster_tools.Coefficient_Variation(FaseBanco)
```

Calcula la variación porcentual en torno a la media en la ley de los bloques de cada cluster de la fase banco (CV). Devuelve el promedio del CV de los clusters y una lista con los valores individuales de CV de cada cluster.

Parámetros

FaseBanco : pandas.DataFrame

Dataframe de la fase banco. Es un sub-dataframe de la mina, cuyos bloques comparten fase y banco (coordenada z).

4 Paralelización

Importante: Estas funciones de paralelización deben estar directamente en el archivo .py donde se realiza la clusterización, como en `test_cluster.py`. Se incluyen al módulo con el fin de copiar y pegar el código.

4.1 Funciones de paralelización del proceso de clusterización

4.1.1 clustering_wrapper

```
cluster_tools.clustering_wrapper(args)
```

Desempaqueta los argumentos necesarios y ejecuta 'Clustering_mina' sobre una fase de la mina.

4.1.2 Clustering_parallel

```
cluster_tools.Clustering_parallel(mina, workers=1, cm=2, cr=0.25, cp=10, P=4, R=0.85,
alpha_ley_corte=0, options=dict())
```

Ejecuta la clusterización por fases en paralelo.

Parámetros:

mina : pandas.DataFrame

DataFrame de la mina

workers : int

Número de procesos a utilizar. Si es -1 , se utilizan todos los núcleos disponibles.

cm : float

Costo mina.

cr : float

Costo recuperación.

cp : float

Costo planta.

P : float

Precio del cobre.

R : float

Recuperación metalúrgica.

alpha_ley_corte : float

Parámetro de interpolación entre la ley de corte marginal (0) y la ley de corte crítica (1).

options : dict

Diccionario con las opciones de guardado 'save' (bool) y 'path_save' (PATH) para guardar los resultados de la clusterización.

Opción	Descripción	Valor predeterminado
BlockWidthX	Ancho de bloque (eje x)	10
BlockWidthY	Ancho de bloque (eje y)	10
area_minima_operativa	Área mínima necesaria para la operación. Se utiliza para determinar la profundidad máxima de la rampa	$\pi \cdot 80^2$
peso_distancia	Peso del criterio distancia en la matriz de similaridad	2
peso_ley	Peso del criterio ley en la matriz de similaridad	0
tolerancia_ley	Tolerancia mínima del criterio ley	0.001
peso_direccion_mineria	Peso del criterio dirección de minería en la matriz de similaridad	0.25
tolerancia_direccion_mineria	Tolerancia mínima del criterio dirección de minería	0.001
penalizacion_destino	Penalización del criterio destino en la matriz de similaridad	0.9
penalizacion_roca	Penalización del criterio tipo de roca en la matriz de similaridad	0.9
penalizacion_c	Penalización del criterio precedencia vertical en la matriz de similaridad	0.5
tamaño_maximo_cluster	Tamaño máximo de los clústers. Puede ser un entero positivo o un float entre 0 y 1, que indica la proporción máxima de clúster respecto al tamaño de la fase banco.	60
tamaño_promedio_cluster	Tamaño promedio estimado de los clústers. Debe ser menor a tamaño_maximo_cluster. Puede ser un entero positivo o un float entre 0 y 1, que indica la proporción promedio de clúster respecto al tamaño de la fase banco.	50
tamaño_minimo_cluster	Tamaño mínimo de los clusters. Usado sólo en Shape Refinement. Puede ser un entero positivo o un float entre 0 y 1, que indica la proporción mínima de clúster respecto al tamaño de la fase banco.	10
tolerancia_tamaño_maximo_cluster	Valor mínimo de tamaño_maximo_cluster	10
tolerancia_tamaño_minimo_cluster	Valor mínimo de tamaño_minimo_cluster	5
Shape_Refinement	Indica qué Shape Refinement se usa. Debe tomar los valores None, 'Tabesh' o 'Modificado'	'Tabesh'
Iteraciones_Shape_Refinement	Iteraciones totales del Shape Refinement	5
save_images	Indica si se grafican y guardan las fases banco clusterizadas	True
path_save_images	PATH donde se guardarán las imágenes	'Clusterizacion/Imagenes'
path_save_params	PATH donde se encuentran parámetros adicionales (cono y rampa)	'Clusterizacion/Params'
show_block_label	Indica si se muestra texto sobre los bloques	True
cmap	Mapa de color entre los disponibles por matplotlib. Vea https://matplotlib.org/stable/users/explain/colors/colormaps.html	'jet'

Table 5: Opciones admitidas por *params*.